Greedy algorithms

Principle

Optimization problems are generally solved through a sequence of steps with a set of choices at each step.
Dynamic programming approaches consider all the possible solutions but deal with the different choices in a neat way to define a not too complex algorithm (e.g. polynomial).

A **greedy algorithm** always makes the choice that looks best at the moment, this is called a *local optimal choice*. Sometimes, this strategy will lead to a global optimal solution but this is not always the case in general. One advantage of such algorithms is to provide an efficient procedure.

Example 1: Activity selection problem (interval scheduling)

Problem definition

Input: A set $S = \{a_1, \ldots, a_n\}$ of activities. Each activity has a start time s_i and a finish time f_i $(0 \leq s_i < f_i < \infty)$.

Output: The largest subset $S' \subseteq S$ of compatible activities. Two activities a_i and a_j are compatible if their time interval do not overlap, ie: $f_j \leq s_i$ or $f_i \leq s_j$.

Applications: memory allocation, supply chain management, scheduling a room for multiple concurrent events, ...

To work on this problem, we will consider a set of 11 activities, defined in the table below and **sorted in monotonically increasing order of finish time**

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Solving the problem

- We need to define a greedy choice: *select the activity compatible with the previously selected activities and having the earliest finish time*.
- This choice leaves the resource available for as many other activities as possible.

GREEDY-ACTIVITY-SELECTOR(s, f)

```
1  n = s.length
2  A = {a_1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {a_m}
7          k = m
8  return A
```

Complexity: $O(n \log n)$ (we need to sort), and $\Theta(n)$ if the activities are already sorted

Result on the previous 11 activity set: $A = \{1, 4, 8, 11\}$.

Question: Does this algorithm always output an optimal solution?

Proving optimality for Greedy Algorithms

Two possible solutions

"Greedy stays ahead" arguments

In a context where the optimal structure property of the considered problem has been justified, we prove that the 1st choice made by the greedy algo is optimal.

Exchange arguments

Show that you can iteratively transform an optimal solution into the solution produced by the greedy algorithm without modifying the cost of the optimal solution.

1st method is used in slides, second in some exercises.

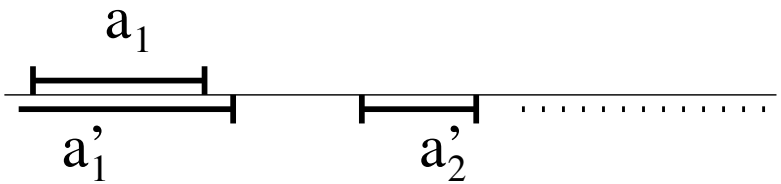
Optimality

Theorem

The solution found by the algorithm is optimal.

Proof: (i) we prove that a_1 belongs to an optimal solution. Let $A = \{a'_1, \ldots, a'_k\}$ be an optimal solution with activities ordered such that $a'_1 < \ldots < a'_n$. If $a_1 = a'_1$ we are done. Otherwise we have:

- $f_{a_1} < f_{a'_1}$ by definition of a_1
- $f_{a'_1} \leq s_{a'_j}$ $\forall j$, because the activities of the solution are compatible



Thus a_1 is compatible with $a'_2, \ldots, a'_k$ and $B = A \cup \{a_1\} - \{a'_1\}$ is a solution with the same size as A, thus B is optimal and contains a_1

Optimality

Theorem

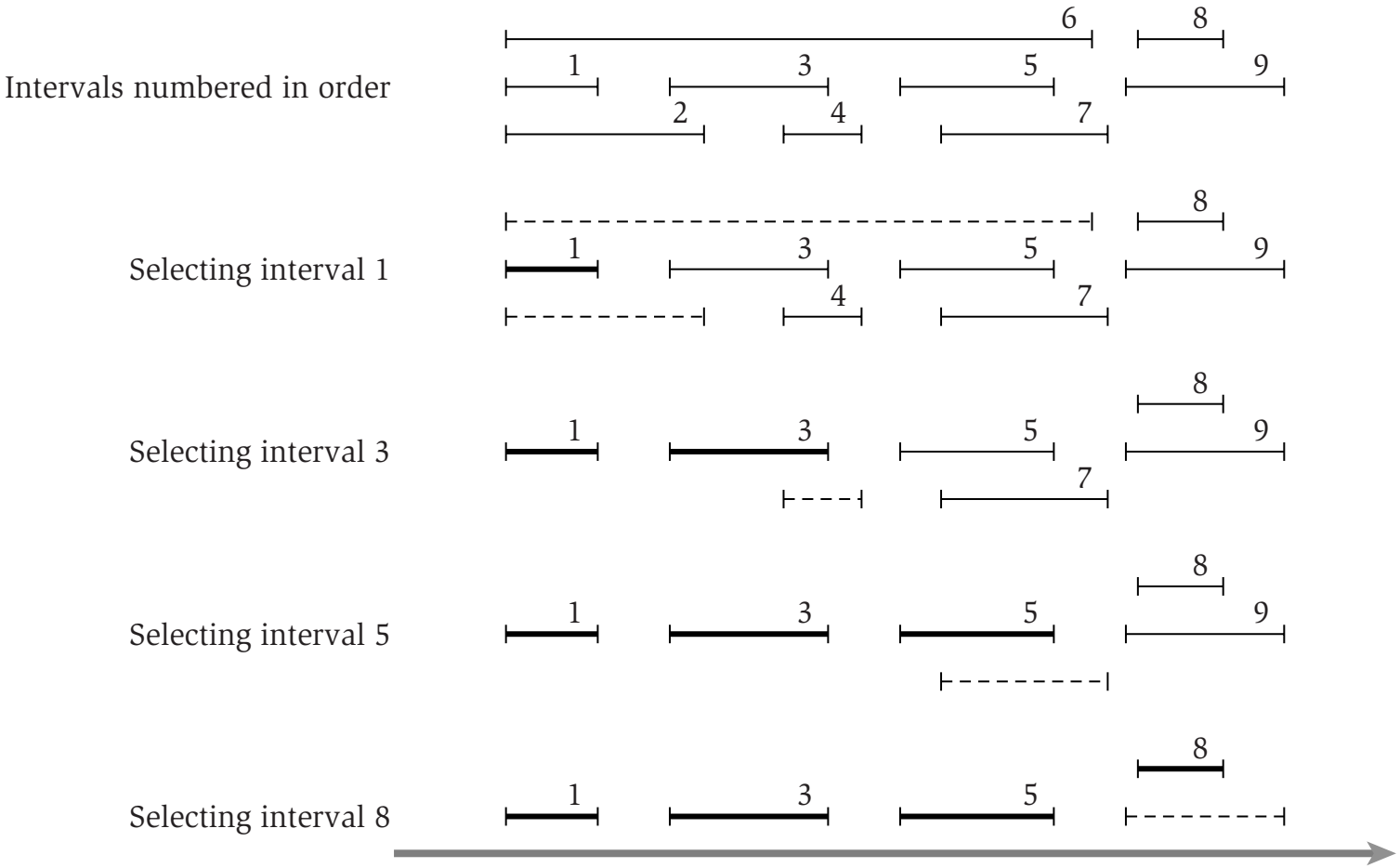
The solution found by the algorithm is optimal

Proof: (ctd)

(ii) Let S_i be the subproblem restricted to the activities a_i to a_n and let A_i be an optimal solution for this subproblem. Clearly, if a is chosen by the algorithm at this step, then $A'_i = A_i - \{a\}$ is optimal for S_{i+1}:
by contradiction assume B' an optimal solution for S_{i+1} and $|B'| > |A'_i|$, then $B = B' \cup \{a\}$ is optimal for S_i with $|B| > |A_i| \rightarrow$ contradiction.$\square$

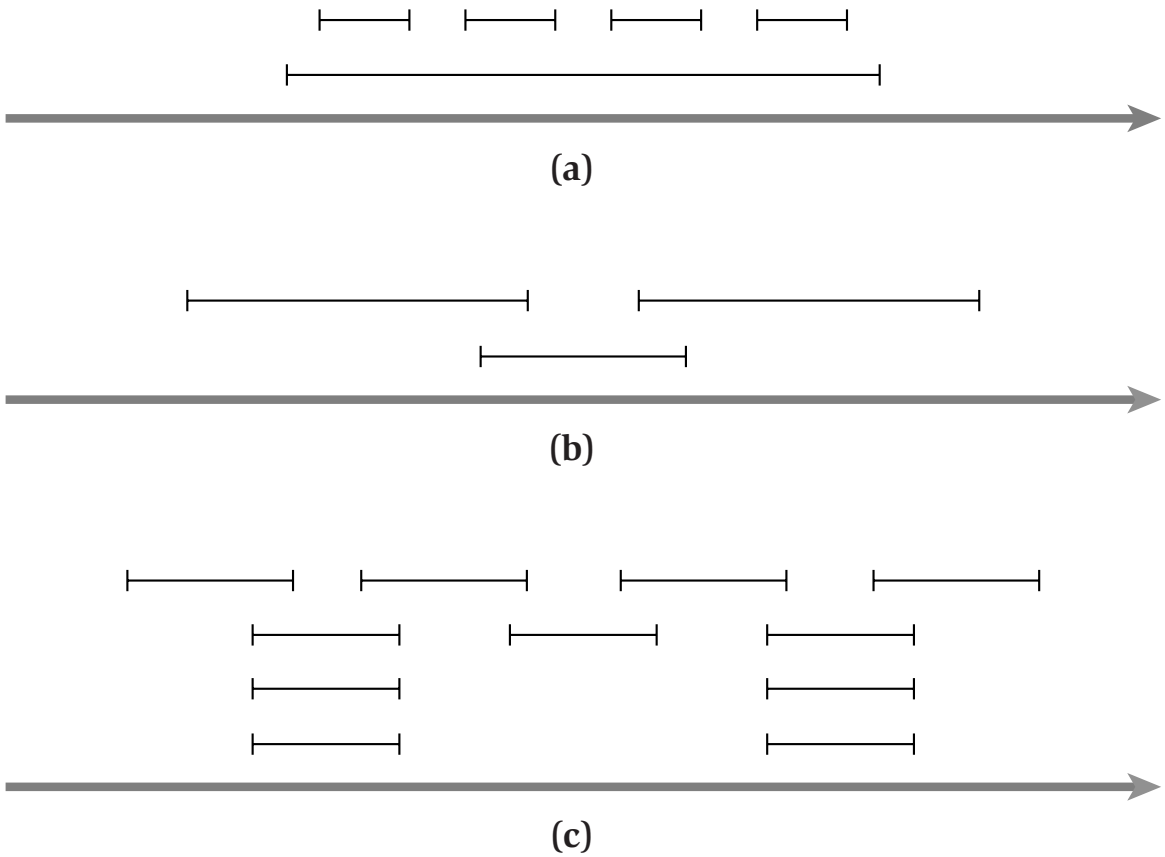
$\Rightarrow a_1$ belongs to an optimal solution and the solutions selected for each subproblem are also in an optimal solution $\rightarrow$ the algo provides an optimal solution.

Example



Sample run of the activity selection (interval scheduling) Algorithm. At each step the selected intervals are darker lines, and the intervals deleted at the corresponding step are indicated with dashed lines.

Why do we need to take the finish time?



Some instances of the activity selection problem on which natural greedy algorithms fail to find the optimal solution. In (a), it does not work to select the interval that starts earliest; in (b), it does not work to select the shortest interval; and in (c), it does not work to select the interval with the fewest conflicts.

What about dynamic programming?

The previous analysis can be seen as a "dynamic programming" one where at each step we have only one subproblem to solve (which is not really relevant for DP). We can make another analysis that really stands into DP scheme but that is unfortunately not so efficient as the greedy approach.

Let S_{ij} be the set of activities starting after a_i and finishing before a_j starts. Let A_{ij} be the maximum set of mutually compatible activities in S_{ij}. Assume $a_k \in A_{ij}$ (a_k belongs to an optimal solution) then we have two subproblems S_{ik} and S_{kj}, $A_{ik} = A_{ij} \cap S_{ik}$ and $A_{kj} = A_{ij} \cap S_{kj}$. Thus $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ and we can easily show that A_{ik} and A_{kj} must be optimal solutions for S_{ik} and S_{kj}.

Then, the recursive definition of an optimal solution is given by $c[i,j] = c[i,k] + c[k,j] + 1$, which leads to the following procedure:

$$c[i,j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset\ , \\ \max\limits_{a_k \in S_{ij}} \{c[i,k] + c[k,j] + 1\} & \text{if } S_{ij} \neq \emptyset\ . \end{cases}$$

(note that $c[i,i] = c[i,i+1] = 0$).
But complexity in $\Omega(n^2)$ and $O(n^3)$!! (other formulations allow one to get $O(n^2)$)

Another example: Huffman code and data compression

We can represent any encoding of a sequence by a tree: The leaves correspond to the letters to encode and the encoding of a letter is obtained by the sequence of labels of the edges from the root to the leaf. Let $d_T(c)$ be the length of the encoding of a letter c, which corresponds to its depth in the encoding tree T.

The problem

Input: a sequence of letters from an alphabet C, each letter has a frequency $f(c)$

Output: a binary encoding for each $c \in C$ of minimal cost, *i.e.* requiring the minimum number of bits to encode the input sequence:

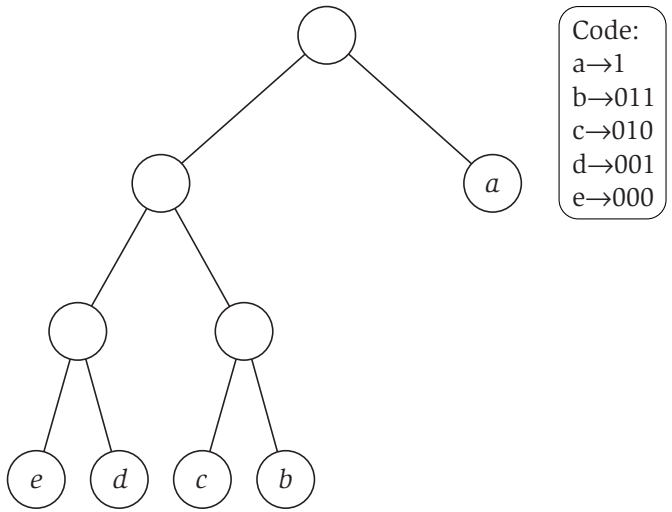
$$B(T) = \sum_{c \in C} f(c) * d_T(c).$$

Encoding

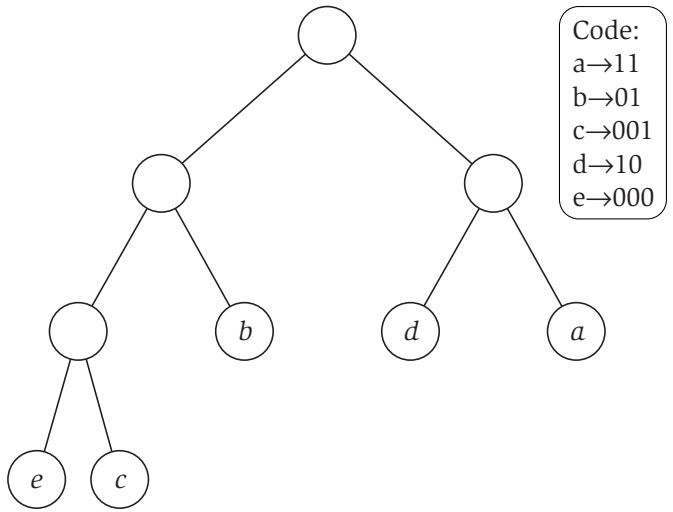
Let $C = \{a, b, c, d, e\}$, consider the following encoding:
$\gamma(a) = 11, \gamma(b) = 01, \gamma(c) = 001, \gamma(d) = 10, \gamma(e) = 000$.
Then *cecab* is encoded as 0010000011101.

Consider a text with frequencies:
$f(a) = 32, f(b) = 25, f(c) = 20, f(d) = 18, f(e) = 5$, using the prefix code we have need $32 * 2 + 25 * 2 + 20 * 3 + 18 * 2 + 5 * 3 = 225$ bits to encode the text.
Using a fixed-length encoding, we would need at least 3 bits per letter, and the 300 bits to encode the text.
There is a better encoding code for this text:
$\gamma(a) = 11, \gamma(b) = 10, \gamma(c) = 01, \gamma(d) = 001, \gamma(e) = 000 \rightarrow 223$ bits.

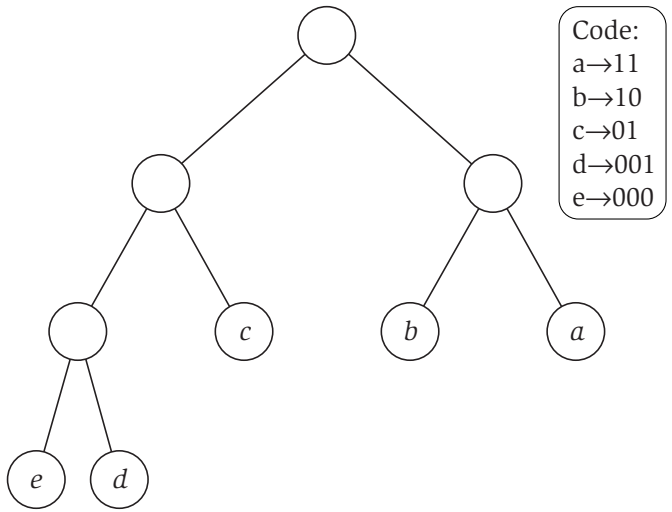
The idea: we will use variable-length codes and to simplify (to avoid ambiguity problems) we consider prefix codes: no codeword is also a prefix of some other codeword (do you see why?). No loss of generality: a prefix code can always achieve the optimal data compression among any character code.


Code:
a→1
b→011
c→010
d→001
e→000
a
e
d
c
b

(a)


Code:
a→11
b→01
c→001
d→10
e→000
b
d
a
e
c

(b)


Code:
a→11
b→10
c→01
d→001
e→000
c
b
a
e
d

(c)

Algorithm

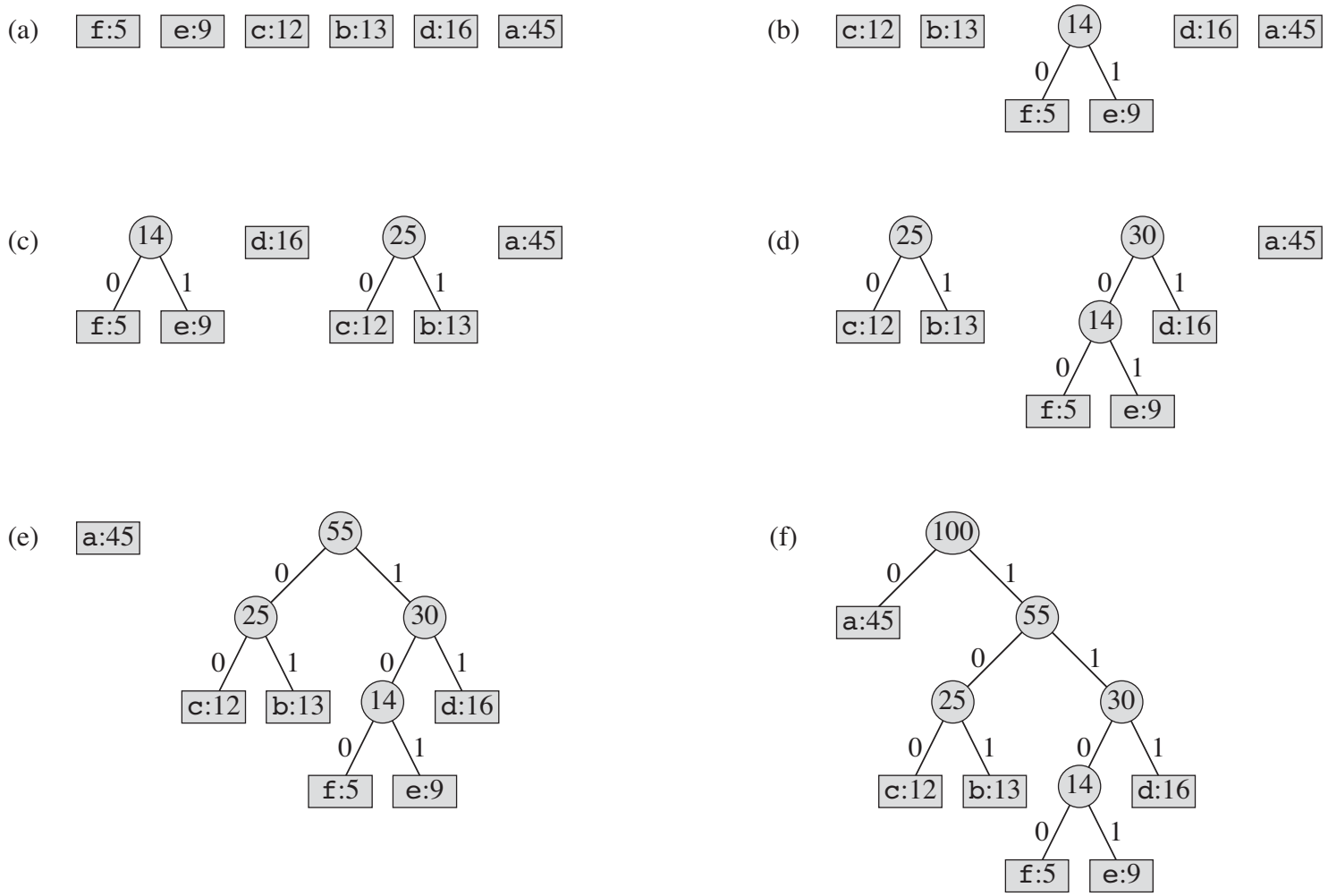
Principle:

- build a leaf for each letter c and associate the value $f(c)$
- Repeat until there remains only one node: merge the two leaves with the lowest frequencies in a node, this node takes the sum of the two leave frequencies as a value.

```
HUFFMAN(C)
1  n = |C|
2  Q = C
3  for i = 1 to n − 1
4       allocate a new node z
5       z.left = x = EXTRACT-MIN(Q)
6       z.right = y = EXTRACT-MIN(Q)
7       z.freq = x.freq + y.freq
8       INSERT(Q, z)
9  return EXTRACT-MIN(Q)      // return the root of the tree
```

We use a min-priority queue (based on the frequencies) to identify the two least frequent objects to merge together.

Example



Using a good data structure (binary heap) EXTRACT-MIN and INSERT require $O(\log n)$, line 2 requires $O(n) \Rightarrow O(n \log n)$.

Optimality

Greedy choice property

Let $x, y \in C$ the two least frequent symbols, there exists an optimal prefix encoding in which the codewords of x and y have the same length and differ only in the last bit. (x and y are two sons of the same node in an optimal tree)

Proof Let T be an optimal tree and let a and b two leaves of maximum depth having the same father node. By assumption $f(x) \leq f(a)$ and $f(y) \leq f(b)$, thus by considering the tree T' obtained by exchanging x and a, we have by definition of x and a:

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c) * d_T(c) - \sum_{c \in C} f(c) * d_{T'}(c) \\ &= f(x)d_T(x) + f(a)d_T(a) - f(x) * d_{T'}(x) - f(a)d_{T'}(a) \\ &= f(x)d_T(x) + f(a)d_T(a) - f(x) * d_T(a) - f(a)d_T(x) \\ &= (f(a) - f(x))(d_T(a) - d_T(x)) \geq 0 \end{aligned}$$

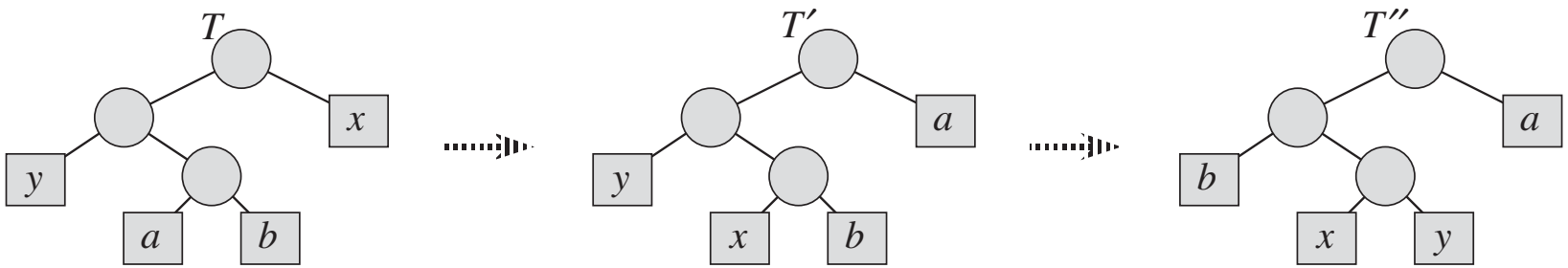
Optimality (2)

Greedy choice property

Let $x, y \in C$ the two least frequent symbols, there exists an optimal prefix encoding in which the codewords of x and y have the same length and differ only in the last bit. (x and y are two sons of the same node in an optimal tree)

Proof (ctd):

We use the same argument for T'' in which y and b have been exchanged: the cost of the tree does not increase $\rightarrow$ there exists an optimal encoding having x and y as maximum depth leaves in the corresponding tree.



Optimal substructure

The next result shows that the problem of constructing optimal prefix codes has the optimal-substructure property.

Let T be a tree of an optimal encoding for C, x, y two maximum depth leaves of the same father z. If $f(z) = f(x) + f(y)$ then $T' = T \backslash \{x, y\} \cup \{z\}$ defines an optimal encoding for $C' = C \backslash \{x, y\} \cup \{z\}$.

Proof: We have for any $c \notin \{x, y\} \in C$: $f(c)d_T(c) = f(c)d_{T'}(c)$ and $d_T(x) = d_T(y) = d_{T'}(z) + 1$. Thus,
$f(x)d_T(x) + f(y)d_T(y) = (f(x) + f(y))(d_{T'}(z) + 1) = f(z)d_{T'}(z) + f(x) + f(y)$ and $B(T) = B(T') + f(x) + f(y)$.
If T' is not optimal, then there exists T'' in which the leaves are the symbols of C' s.t. $B(T'') < B(T')$. By adding x, y to the leaf z of T'', we have a prefix encoding for C with cost $B(T'') + f(x) + f(y) < B(T)$, which contradicts that T is optimal $\rightarrow$ T' is necessarily optimal for T'.$\square$

$\Rightarrow$ By combining the two results, we have that the Huffman algorithm produces an optimal prefix code.

Greedy algorithms - Summary and Take home message

To solve an optimization problem **exactly** by a greedy approach, two properties are necessary:

- Greedy choice property: the local choice made at each step of the algorithm belongs to an optimal solution.
- Optimal substructure property: the optimal solution can be decomposed into optimal solution of a subproblem
- A greedy algorithm must have a "small complexity", local greedy choices must be done in constant time (possibly) or in linear time (in the most general cases).